# Islamic University of Technology


Visual Programming Lab

CSE 4402

**Final Project Report**

---

# GariAche

---

**Team: BuggyRiders**

**Supervisors**

Sabrina Islam

Mohammad Ridwan Kabir

**Team Members**

Rahinur Bin Naushad - 220041118

Fahim Tazoar - 220041140

Md. Zarif Bin Hasnat - 220041144

September 30, 2025

**GariAche - IUT Carpooling Android App**

GariAche is a mobile carpooling application designed exclusively for the stu- dents of the Islamic University of Technology (IUT). It provides a platform for students to offer and find rides, primarily between the IUT campus and various destinations in Dhaka, fostering a sense of community and providing a cost-effective transportation solution.

GitHub repo: GariAche

**Core Features**

- **User Authentication** : Secure sign-up and login system restricted to users with an @iut-dhaka.edu email address.

- **Offer a Ride** : Allows users (drivers) to post ride details, including destination, time, available seats, and price.

- **Find a Ride** : Enables users (passengers) to search for available rides based on their destination and date.

- **Ride Booking** : A seamless process for passengers to book one or more seats in an available ride.

- **Booking Management** : Users can view their booked rides and have the option to cancel a booking up to 30 minutes before the departure time.

- **User Profile** : Displays user information, confirming their status as a verified IUT student.

- **Dynamic UI** : The interface dynamically updates to prevent users from booking their own rides or rides that are already full.

**Technology Stack & Key Concepts**

This project was built using native Android development with Java, leveraging fundamental Android components and a local SQLite database for data persistence.

- **Language** : **Java**

- **Database** : **SQLite -** A localSQLiteOpenHelperclass is used to manage the database schema (creation, upgrades). All user data, rides, and bookings are stored locally on the device.

- **UI/UX** :

  **- Android XML Layouts** : All user interfaces are built using XML.

  **- Material Design Components** : The UI is styled using components from the Material Design library (CardView,TextInputEditText, MaterialAutoCompleteTextView,Button) for a modern and intuitive user experience.

  **- RecyclerView** : Used efficiently to display lists of rides and bookings. This is a key component for performance with dynamic lists.

  **- Dynamic View Inflation** : Search results and ride lists are created by programmatically inflating item layouts (ride_item.xml, booking_item.xml) and adding them to a parent container or RecyclerView.

- **Core Android Components** :

  **- Activities** : The application follows a multi-activity architecture, with each screen representing a distinct task.

  **- Intents** : Used for navigation between activities and for passing data (e.g., ride details for booking).

  **- SharedPreferences** : Used for lightweight session management, specifically for storing thecurrent_user_idafter a successful login.

  **- Dialogs** : DatePickerDialog,TimePickerDialog, and AlertDialog are used for user input and confirmations, providing a native and user-friendly experience.

- **Architecture** :

  **-** The application employs a standard **Activity-based architecture**. Each activity is responsible for managing its own UI and business logic.

  **-** Data is passed between activities via Intent extras.

  **-** A centralDatabaseHelperclass manages all database interactions, separating database logic from the UI controllers (Activities).

**Database Schema**

The application uses a local SQLite database named GariAche.db with three main tables.

1. **users**

    - id(INTEGER, Primary Key, Autoincrement)
    - student_id(TEXT, UNIQUE, NOT NULL)
    - name(TEXT, NOT NULL)
    - email(TEXT, UNIQUE, NOT NULL)
    - phone(TEXT)
    - password(TEXT, NOT NULL)

2. **rides**

    - id(INTEGER, Primary Key, Autoincrement)
    - driver_id(INTEGER, Foreign Key tousers.id)
    - pickup_point(TEXT)
    - destination(TEXT, NOT NULL)
    - ride_date(TEXT, NOT NULL)
    - ride_time(TEXT, NOT NULL)
    - available_seats(INTEGER, NOT NULL)
    - price_per_seat(INTEGER)
    - car_model(TEXT)
    - car_color(TEXT)
    - license_plate(TEXT)
    - notes(TEXT)
    - status(TEXT, e.g., ‘active’)

3. **bookings**

    - id(INTEGER, Primary Key, Autoincrement)
    - ride_id(INTEGER, Foreign Key torides.id)
    - passenger_id(INTEGER, Foreign Key tousers.id)
    - booked_seats(INTEGER, NOT NULL)
    - passenger_phone(TEXT, NOT NULL)
    - notes(TEXT)
    - booking_date(TEXT)
    - status(TEXT, e.g., ‘confirmed’, ‘cancelled’)

**Activity & Logic Breakdown**

**1. SplashActivity**

- **Purpose** : The initial entry point of the app, displaying the app logo and name.

- **UI (activity_splash.xml)** : A simple centered layout with an ImageViewfor the logo andTextViews for the app name and tagline.

- **Logic (SplashActivity.java)** :

  **- Technique** : Uses a Handler().postDelayed() method to create a short delay (500ms).

  **-** After the delay, it creates an Intent to navigate to LoginActivity and then calls finish() to remove the splash screen from the back stack.

**2. LoginActivity**

- **Purpose** : To handle user authentication (both login and sign-up).

- **UI (activity_login.xml)** :

  **-** Features two distinct forms (LinearLayout) for Login and Sign-Up.

  **-** The visibility of these forms is toggled programmatically, providing a clean "single screen" experience for the user.

- **Logic (LoginActivity.java)** :

  **- Mode Toggling** : A boolean isLoginMode flag tracks the current state. A TextView(toggle_text) acts as a button to switch between login and sign-up modes by changing text and form visibility.

  **- Sign-Up (handleSignup)** :

    - **Validation** : Enforces that all fields are filled and that the email address ends with @iut-dhaka.edu.

    - **Database Interaction** : Queries theuserstable to ensure the email or student ID does not already exist before inserting a new user.

**- Login (handleLogin)** :

- **Database Interaction** : Queries theuserstable for a matching email.
- **Authentication** : Compares the provided password with the stored password from the database.
- **Session Management** : On successful login, the user's id is retrieved from the database and stored inSharedPreferences under the keycurrent_user_id. This acts as the session token for the rest of the app.
- **Navigation** : Redirects to HomeActivity on success.

**3. HomeActivity**

- **Purpose** : The main dashboard or home screen after a user logs in.
- **UI (activity_home.xml)** : A clean, card-based layout.

  **-** Two primary actions, "Find a Ride" and "Offer a Ride", are presented as large, clickableCardViews.

  **-** Secondary actions like "My Profile", "My Bookings", and "Logout" are provided as TextViews at the bottom.
- **Logic (HomeActivity.java)** :

  **- Authentication Check** : In onCreate(), it immediately checks SharedPreferences for current_user_id. If not found, it redirects the user back to LoginActivity.

  **- Navigation** : Uses setOnClickListener on the cards and text views to create Intents that navigate to the respective activities (FindRideActivity,OfferRideActivity, etc.).

  **- Logout** : The logout button clears all data from SharedPreferences and navigates back to LoginActivity, effectively ending the user's session.

**4. OfferRideActivity**

- **Purpose** : Allows drivers to create and post a new ride.

- **UI (activity_offer_ride.xml)** : A comprehensive form built with TextInputEditTextfor ride details.

  **-** The "Pickup Point" is fixed to "IUT" and is not editable.

  **-** A MaterialAutoCompleteTextView is used for the "Destina- tion," providing a dropdown list of predefined locations from res/values/string.xml.

- **Logic (OfferRideActivity.java)** :

  **- User Input** : Uses DatePickerDialog and TimePickerDialog to ensure valid date and time formats.

  **- Validation** : The handlePostRide() method performs extensive validation to ensure all required fields are filled correctly.

  **- Database Insertion** : On successful validation, it retrieves the current_user_id from SharedPreferences and creates a new record in the rides table in the SQLite database.

  **- Navigation** : On success, it passes the new ride's details to OfferRideSuccessActivity via Intent extras.

**5. FindRideActivity**

- **Purpose** : Allows passengers to search for available rides.

- **UI (activity_find_ride.xml)** :

  **-** A search form for destination and date.

  **-** A NestedScrollView contains a Linear Layout(rides_container) where search results are dynamically added.

  **-** A "No rides found" message is shown if the search yields no results.

- **Logic (FindRideActivity.java)** :

  **- Database Search** : The searchRidesInDatabase() method constructs and executes a SQLSELECT query on the rides table with WHERE clauses for destination, date, and status.

**- Dynamic UI Generation** : The displaySearchResults() method iterates through the Cursor results. For each ride, it inflates the ride_item.xml layout, populates its TextViews with ride data, and adds the view to the rides_container.

**- Conditional Logic** :

- It fetches the current_user_id from SharedPreferencesto check if a ride is owned by the current user. If so, the "Book" button is disabled and styled differently.
- It also disables the button if available_seats is 0.

**- Navigation** : The "Book" button onClicklistener creates an Intent to BookingConfirmationActivity, passing all the necessary ride details.

**6. BookingConfirmationActivity**

- **Purpose** : To allow a user to confirm the booking of a selected ride.
- **UI (activity_booking_confirmation.xml)** :

  **-** Displays the details of the ride passed from the previous activity.

  **-** Provides input fields for the number of seats and phone number.

  **-** A "Total Amount" TextViewis updated dynamically.

- **Logic (BookingConfirmationActivity.java)** :

  **- Data Reception** : Retrieves ride details from the Intent extras.

  **- Dynamic Price Calculation** : A setOnFocusChange Listener is attached to the seats input field. When the user finishes editing the number of seats, updateTotalPrice() is called to calculate and display the total cost (price * seats).

  **- Booking Logic (handleBookingConfirmation)** :

  1. Validates the user's input.
  2. Retrieves current_user_id from SharedPreferences.
  3. Inserts a new record into the bookings table.

4. Executes an UPDATE query on the rides table to decrement the number of available_seats.

5. Navigates to BookingSuccessActivity on success.

**7. ViewMyRidesActivity & BookingsAdapter**

- **Purpose** : To display a list of all rides the current user has booked.

- **UI (activity_view_my_rides.xml)** : The main element is a RecyclerView to efficiently handle a potentially long list of bookings.

- **Logic (ViewMyRidesActivity.java)** :

  **- Data Fetching** : The loadMyBookings() method executes a complex SQLJOINquery to fetch booking details along with the corre- sponding ride and driver information from the bookings, rides, and users tables.

  **-** The results are stored in a list of BookingWithRideInfo data class objects.

- **Adapter Logic (BookingsAdapter.java)** :

  **- View Binding** : Binds the BookingWithRideInfo data to the views inside booking_item.xml within the onBindViewHolder method.

  **- Cancellation Logic** :

  - The canCancelBooking() method calculates the time difference between the current time and the ride's departure time. It returns true if the difference is more than 30 minutes.

  - The "Cancel" button's state (enabled/disabled) and text are set based on the result of this method and the booking status.

  - If clicked, an AlertDialog is shown for confirmation. If con- firmed, the cancelBooking() method updates the booking's status to "cancelled" in the database and notifies the adapter to refresh the view.

**8.ProfileActivity**

- **Purpose** : To display the profile information of the logged-in user.

- **UI (activity_profile.xml)** : A simple, clean layout that displays the user's name, email, phone, and student ID.

- **Logic (ProfileActivity.java)** :

  - Retrieves the current_user_id from SharedPreferences.

  - Queries the users table using this ID to fetch the user's details.

  - Populates the TextViews with the retrieved information.

**Class Diagram:**